GPU-NFBench: A Human-Labeled Benchmark and LLM-Assisted Triage System for GPU Numerical Failures

Aryan Shah
Independent Researcher
Texas, USA

Abstract—GPU software stacks expose high-performance kernels through Python APIs, array libraries, graph compilers, and domain-specific languages, but numerical failures in these systems are difficult to diagnose from issue reports alone. Reports about NaNs, overflow, dtype conversion, precision tolerance, and incorrect outputs often mix user error, API limitations, compiler failures, performance regressions, and true numerical defects. This paper presents GPU-NFBench, a human-labeled benchmark and triage framework for GPU numerical failure reports. We construct an expanded 1,191-issue gold benchmark from public GitHub issues across eight GPU-related repositories, using a seven-class taxonomy that separates invalid-value failures, overflow/underflow, precision/tolerance mismatches, dtype/casting failures, crash/compile failures, performance-only reports, and non-numerical reports. We also preserve the original 930-issue silver seed, the 191-issue adjudicated pilot benchmark, and a full taxonomy-normalization log for the expanded labels. On the expanded gold benchmark, weak candidate labels reach only 50.5% accuracy, showing that keyword retrieval is not sufficient ground truth. A deterministic full-coverage voting ensemble reaches 73.8% accuracy and 0.712 macro F1, while a fine-tuned standalone FLAN-T5-base classifier reaches 80.5% held-out test accuracy and 0.778 macro F1. Under leave-one-repository-out evaluation, the best deterministic models reach 66.8% accuracy, showing that project transfer remains difficult. In a chronological split that trains on older reports and tests on newer reports, the best deterministic model reaches 76.6% accuracy and 0.738 macro F1. Selective prediction further improves precision: the deterministic ensemble reaches 84.2% accuracy at 73.5% coverage and 93.9% accuracy at 30.1% coverage, while an LLM/deterministic agreement-abstention mode reaches 89.3% accuracy at 68.3% coverage. A 119-row blind audit achieved 80.7% Person A/B agreement and Cohen's $\kappa = 0.721$; follow-on adjudication applied 419 human label updates, changing 211 primary labels and producing the canonical v2 benchmark used in the final experiments. We also provide a 200-row root-cause evidence-coded extension, including a 50-row human-adjudicated subset, and an appendix of concrete boundary errors. These results position GPU-NFBenchas a practical foundation for GPU-bug triage, dataset construction, and future work on root-cause prediction for numerical failures in ML systems.

Index Terms—GPU bugs, numerical bugs, benchmark construction, issue mining, LLM-assisted triage, software reliability, machine learning systems

I. Introduction

Modern machine-learning and scientific-computing workflows rely on GPU software stacks that combine low-level kernels, compiler transformations, and high-level Python interfaces. Frameworks such as Triton, PyTorch, JAX, CuPy, Numba, RAPIDS, and TVM make GPU acceleration accessible, but they also create a large surface for numerical failures. A small dtype mismatch, missing mask, incorrect compiler lowering rule, or precision-tolerance assumption can produce NaNs, Inf values, silent wrong answers, or failures that appear only on specific hardware and software configurations.

Public issue trackers contain a large record of how these failures occur in practice. However, issue reports are noisy. A search for "NaN" can return true invalid-value bugs, missing-data examples, or unrelated benchmark tables. A search for "dtype" can return casting failures, unsupported-feature requests, performance complaints, or compiler crashes. These reports are valuable, but only if the benchmark distinguishes candidate retrieval labels from human gold labels.

This paper presents GPU-NFBench, an expanded human-labeled benchmark for GPU numerical failure reports. The benchmark is designed for practical triage: given a public issue report, classify the dominant failure mode, determine whether the issue is truly numerical, and optionally abstain when the evidence is ambiguous. The artifact also preserves provenance for each stage: raw issue collection, silver labels, pilot adjudication, expanded human labels, label normalization, model training files, and evaluation reports.

The contributions are:

- We construct an expanded 1,191-row human-labeled benchmark of GPU numerical-failure reports from public GitHub issues across eight GPU-related projects.
- We define a seven-class primary taxonomy and preserve a complete normalization log for 180 out-of-taxonomy labels in the expanded human annotation file.
- We evaluate weak labels, lexical baselines, nearest-neighbor retrieval, linear classifiers, a standalone fine-tuned FLAN-T5-base model, and a deterministic voting ensemble under stratified and leave-one-repository-out settings.
- We show that deterministic full-coverage triage

reaches 73.8% accuracy and 0.712 macro F1, while a standalone fine-tuned LLM reaches 80.5% held-out test accuracy and 0.778 macro F1.
- We evaluate selective triage: deterministic abstention reaches 84.2% accuracy at 73.5% coverage, and an LLM/deterministic agreement mode reaches 89.3% accuracy at 68.3% coverage.
- We release a benchmark packet, data card, ablation tables, error analysis, linked-fix evidence, a completed blind audit, a 419-row adjudication update, a 50-row human-adjudicated root-cause subset, and a 200-row evidence-coded root-cause extension.

II. Background and Motivation

GPU numerical bugs differ from ordinary application bugs because the symptom is often detached from the underlying cause. A NaN in a model output can originate from overflow in a fused operation, an unsafe dtype cast, an invalid memory access, a compiler transformation, or a tolerance mismatch in the test suite. Issue reports rarely state the root cause directly. Instead, they contain partial evidence: a stack trace, a failing kernel, a device name, an expected CPU result, a tolerance comparison, or a maintainer comment.

This makes the problem a good testbed for LLM-assisted software engineering. Large language models can read issue text and infer likely failure categories, but they also need calibrated taxonomies, strong negative classes, and evaluation against human labels. In early experiments on the 191-issue pilot benchmark, full-coverage models remained near 54% accuracy, while selective answering performed much better. The expanded benchmark tests whether adding a larger human-labeled gold set makes full-coverage triage practical.

III. Benchmark Construction

A. Seed Collection

The project began with a 930-issue silver seed collected from six GPU-related repositories: Triton, CuPy, PyTorch, JAX, Numba, and RAPIDS cuDF. Queries used terms associated with numerical failure reports, including nan, inf, overflow, dtype, precision, tolerance, incorrect, and wrong result. Candidate labels were assigned from query and text heuristics. This seed was useful for coverage but was not treated as ground truth.

B. Pilot Gold Benchmark

From the seed, we constructed a 191-issue pilot gold benchmark with full public issue/comment context. Two blind human annotation passes labeled primary failure class, secondary causes, numerical-failure status, confidence, and evidence snippets. Disagreements were adjudicated into final gold labels. The pilot revealed two important facts: issue reports are genuinely ambiguous, and weak labels are too noisy to serve as final supervision. Prior agreement analysis on this pilot found modest overall primary-label agreement, but substantially higher agreement on high-confidence cases. This motivated the larger expanded benchmark and the selective-prediction setting.

TABLE I
Expanded gold benchmark label distribution.

Primary label	Issues
precision_tolerance	368
dtype_casting	193
crash_compile	192
not_numerical_failure	147
nan_inf	143
overflow_underflow	94
performance_only	54
Total	1191

C. Expanded Human-Labeled Benchmark

We expanded the benchmark with 1,000 additional completed human-labeled issue records from an online collection queue. The expanded queue covers eight repositories: JAX, PyTorch, RAPIDS cuML, Triton, RAPIDS cuDF, CuPy, Numba, and Apache TVM. After merging the 1,000 expansion rows with the 191 pilot rows and applying the v2 adjudication updates described below, the canonical expanded gold benchmark contains 1,191 issues. Table I shows the primary-label distribution.

D. Taxonomy Normalization

The expanded human annotation file included several labels outside the final seven-class taxonomy, including API feature requests, compiler/code-generation descriptions, generic needs-review labels, and performance-regression labels. Rather than silently discard or recode them, we ran a deterministic taxonomy-normalization pass and preserved the full change log. In total, 180 rows were mapped into the final taxonomy, and no invalid labels remained. For example, performance-regression labels were mapped to performance_only; compiler/code-generation labels were mapped to crash_compile, performance_only, or not_numerical_failure depending on the recorded evidence; and API feature requests were mapped to not_numerical_failure. The original labels remain recoverable from the repair log, so this step is auditable.

E. Expanded Agreement Audit and V2 Canonicalization

The expanded 1,000-row addition was followed by a blind audit sampled from the expansion set. The audit packet hides gold labels, candidate labels, model predictions, and repair decisions. Two annotators independently labeled 119 rows using the final seven-class taxonomy. Person A and Person B agreed on 80.7% of rows, with Cohen's $\kappa = 0.721$, indicating substantial agreement for a report-level taxonomy with overlapping symptom and root-cause cues. A third adjudication pass then assigned

final labels to all 119 audit rows. The adjudicated audit differed from the existing expanded gold label in 52 rows, so we used it as a targeted revision signal rather than treating the expanded labels as fixed. We then completed a second 300-row adjudication pass focused on low-confidence and model-error examples. Together, the audit and follow-on adjudication applied 419 human updates to the expanded benchmark and changed 211 primary labels. The final experiments in this paper use this canonical v2 benchmark. This revision step is important methodologically: it turns disagreement analysis into benchmark repair rather than reporting label noise as an unresolved limitation.

IV. Taxonomy

Each issue receives exactly one primary label:

- nan_inf: invalid-value failures involving NaN, Inf, or non-finite outputs.
- overflow_underflow: numerical range failures, including saturation, wraparound, or loss to zero.
- precision_tolerance: wrong-answer or tolerance failures where approximate arithmetic, accumulation order, or precision is central.
- dtype_casting: dtype promotion, conversion, unsupported dtype, or casting semantics failures.
- crash_compile: failures dominated by crashes, compiler errors, illegal memory accesses, or build/runtime exceptions.
- performance_only: reports about speed, memory use, or scheduling where no numerical correctness issue is present.
- not_numerical_failure: feature requests, API questions, documentation issues, and other non-numerical reports.

The taxonomy intentionally separates symptoms from suspected causes. For example, an issue may be labeled nan_inf as the primary symptom while secondary evidence points to dtype conversion, masking, or compiler lowering. This paper evaluates primary-label triage; secondary-cause prediction is left as a follow-up task.

V. Models and Evaluation

A. Evaluation Setup

We evaluate models on the 1,191-row expanded gold benchmark. Metrics are accuracy and macro F1 for the seven-class task. Macro F1 is important because the dataset is imbalanced: precision/tolerance issues are the largest class, while performance-only reports are the smallest. We also evaluate a binary gate that predicts whether an issue is a numerical failure at all. The primary headline setting is stratified five-fold evaluation. We also report leave-one-repository-out evaluation, where each repository is held out as the test project and all other repositories are used for training.

B. Baselines

The majority baseline predicts the largest class. The weak-label baseline uses the original candidate label attached during retrieval. Text baselines include BM25 nearest-neighbor retrieval, multinomial naive Bayes, TF-IDF logistic regression, TF-IDF linear SVM, and bigram TF-IDF logistic regression. These baselines are deliberately simple so that improvements can be attributed to better supervision and calibrated decision rules rather than opaque modeling choices.

C. Standalone LLM

We fine-tuned a standalone FLAN-T5-base sequence-to-sequence classifier on benchmark-formatted issue text from the v2 canonical benchmark. The split contains 945 training rows, 123 validation rows, and 123 held-out test rows. To reduce class imbalance, the training set was balanced by oversampling minority classes to 2,058 training examples. The selected v2 checkpoint starts from the previous expanded-gold FLAN-T5-base checkpoint and is retrained for one epoch on the v2 labels. It predicts one of the seven primary taxonomy labels directly from issue text.

D. Voting Ensemble and Abstention

The best full-coverage deterministic system is a voting ensemble over the strongest lexical and linear models. For selective triage, the ensemble can abstain unless at least k voters agree. This setting matches realistic engineering use: a triage assistant should confidently label straightforward issues and flag ambiguous reports for human review.

We also evaluate an LLM-assisted agreement mode. This mode answers only when the deterministic ensemble and standalone LLM predict the same label; otherwise it abstains. This is stricter than majority voting and is intended for high-confidence triage queues where false labels are more costly than deferred labels.

E. Ablations and Error Analysis

To test whether the ensemble depends too heavily on candidate labels, we report ablations that remove candidate-label votes and retain only learned text models. We also compute the most frequent gold/predicted error pairs and retain representative issue examples for qualitative review.

VI. Results

A. Full-Coverage Classification

Table II reports full-coverage performance. Weak candidate labels reach only 50.5% accuracy, confirming that keyword retrieval labels are not reliable gold labels. Linear text models perform substantially better with the expanded human-labeled set. The full-coverage voting ensemble reaches 73.8% accuracy and 0.712 macro F1.

TABLE II
Expanded gold benchmark full-coverage results.

Model	Accuracy	Macro F1
Majority baseline	0.309	0.067
Candidate weak label	0.505	0.426
BM25 nearest neighbor	0.571	0.534
Naive Bayes	0.610	0.501
TF-IDF logistic regression	0.715	0.688
TF-IDF linear SVM	0.732	0.703
Bigram TF-IDF logistic	0.724	0.694
Voting ensemble	0.738	0.712

TABLE III
Leave-one-repository-out evaluation on expanded gold.

Model	Accuracy	Macro F1
Candidate weak label	0.505	0.426
BM25 nearest neighbor	0.493	0.478
Naive Bayes	0.550	0.475
TF-IDF logistic regression	0.653	0.630
TF-IDF linear SVM	0.668	0.648
Bigram TF-IDF logistic	0.663	0.633
Voting ensemble	0.668	0.647

B. Cross-Repository Generalization

Table III reports leave-one-repository-out results. This setting is harder because model vocabularies, issue templates, and library-specific concepts shift across projects. The best deterministic models in this setting reach 66.8% accuracy, with TF-IDF linear SVM reaching 0.648 macro F1 and the voting ensemble reaching 0.647 macro F1. The result is substantially above the weak-label baseline but lower than shuffled evaluation, indicating that cross-project transfer remains a central challenge for GPU numerical-failure triage.

Table IV summarizes the weakest transfer repositories. CuPy and Numba are the hardest held-out projects: CuPy mixes dtype/API compatibility with numerical symptoms, while Numba contains CUDA, LLVM, build, and runtime language that blurs crash_compile and not_numerical_failure. This supports cross-repository transfer as a real benchmark challenge rather than a simple shuffled-text classification task.

C. Chronological Generalization

We also evaluate a future-facing chronological split. Issues are sorted by creation time; the older 80% form the training set and the newer 20% form the test set. The split trains on 952 issues through December 17, 2025 and tests on 239 newer issues. Table V shows that bigram TF-IDF logistic regression reaches 76.6% accuracy and 0.738 macro F1, while the voting ensemble reaches 74.5% accuracy and 0.722 macro F1. These results suggest that the benchmark supports triage of later issue reports, but also that the simpler linear model transfers better than the ensemble under temporal shift.

TABLE IV
Weakest leave-one-repository-out transfer cases.

Held-out repo	Issues	Best acc.	Ensemble acc.
numba/numba	65	0.492	0.462
cupy/cupy	82	0.500	0.439
rapidsai/cudf	132	0.583	0.545

TABLE V
Chronological 80/20 evaluation on expanded gold.

Model	Accuracy	Macro F1
Candidate weak label	0.469	0.325
Naive Bayes	0.598	0.475
TF-IDF logistic regression	0.711	0.690
TF-IDF linear SVM	0.732	0.697
Bigram TF-IDF logistic	0.766	0.738
Voting ensemble	0.745	0.722

D. Standalone LLM Performance

Table VI compares the standalone LLM with the best deterministic ensemble. The v2 standalone FLAN-T5-base model reaches 80.5% held-out test accuracy and 0.778 macro F1, outperforming the deterministic ensemble on the same held-out LLM test split. This changes the practical conclusion from the earlier expanded-gold version: after adjudication repair, the standalone LLM is not only a simpler deployable artifact, but also the strongest full-coverage triage model in the final held-out evaluation. As an additional local zero-shot comparison, we evaluated Ollama llama3.2:3b on the same 123 held-out rows. It reaches only 31.7% accuracy and 0.322 macro F1, showing that task-specific fine-tuning is important. A cloud-backed modern model was not used for the final comparison because exporting held-out benchmark prompts to an external service was blocked in the local environment.

E. Selective Triage

Table VII shows the accuracy/coverage tradeoff when the ensemble abstains unless enough voters agree. With at least three agreeing voters, the system labels 92.6% of issues at 76.9% accuracy. With at least four agreeing voters, it labels 73.5% of issues at 84.2% accuracy. With unanimous or near-unanimous agreement, it reaches 93.9% accuracy on 30.1% of issues. The LLM/deterministic agreement mode provides a complementary high-confidence setting: it answers 68.3% of held-out LLM-test examples at 89.3% accuracy and 0.844 macro F1.

F. Label-Signal Check

The v2 results show that models are not merely replaying weak retrieval labels. The candidate weak-label baseline reaches 50.5% accuracy and 0.426 macro F1, while the TF-IDF linear SVM reaches 73.2% accuracy and 0.703 macro F1 and the standalone LLM reaches 80.5% held-out accuracy and 0.778 macro F1. The improvement from weak labels to text-trained models indicates that

TABLE VI
Standalone LLM and deterministic comparison on v2 labels.

Model	Accuracy	Macro F1
Local Llama 3.2 3B zero-shot	0.317	0.322
TF-IDF linear SVM on LLM test split	0.691	0.669
Voting ensemble on LLM test split	0.683	0.654
FLAN-T5-base standalone LLM	0.805	0.778
Binary numerical gate, 5-fold	0.892	0.796

TABLE VII
Selective prediction with abstention.

Agreement rule	Coverage	Accuracy	Macro F1
Vote ≥ 2	0.999	0.739	0.713
Vote ≥ 3	0.926	0.769	0.744
Vote ≥ 4	0.735	0.842	0.803
Vote ≥ 5	0.301	0.939	0.733
LLM/ensemble agreement	0.683	0.893	0.844

the human-labeled benchmark provides usable semantic supervision beyond query terms or retrieval heuristics.

G. Per-Class Behavior

The deterministic ensemble is strongest on precision_tolerance, nan_inf, and crash_compile issues, reaching F1 scores of 0.811, 0.769, and 0.740, respectively. It is weakest on performance_only, not_numerical_failure, and dtype_casting. These classes are boundary-heavy: performance reports often include numerical benchmark terms, non-bug support questions can contain stack traces or dtype language, and dtype changes can produce precision or range symptoms.

H. Error Analysis

The largest error families are semantically plausible boundary cases. The most common confusion is dtype_casting predicted as precision_tolerance (28 errors), followed by nan_inf predicted as precision_tolerance (22 errors), precision_tolerance predicted as dtype_casting (21 errors), and crash_compile predicted as precision_tolerance (20 errors). These errors reflect the structure of GPU numerical reports: dtype changes, compiler failures, overflow, and NaN symptoms are often reported through failed numerical tolerance tests. The next largest errors involve not_numerical_failure issues predicted as crash or dtype failures, usually because issue text contains API type errors or installation stack traces even when the report is not a numerical defect. This analysis supports two follow-up tasks: multi-label secondary-cause prediction and explicit separation of symptom labels from root-cause labels.

The artifact includes a 12-case error appendix with issue snippets and boundary explanations. Examples include dtype reports predicted as precision/tolerance because the failure appears as a numerical mismatch, NaN/Inf

TABLE VIII
Voting ensemble per-class performance.

Class	Precision	Recall	F1
crash_compile	0.725	0.755	0.740
dtype_casting	0.641	0.694	0.667
nan_inf	0.855	0.699	0.769
not_numerical_failure	0.689	0.619	0.652
overflow_underflow	0.813	0.649	0.722
performance_only	0.744	0.537	0.624
precision_tolerance	0.761	0.867	0.811

reports embedded in broader tolerance failures, and non-bug support requests predicted as crash or dtype failures because they contain stack-trace or type-language cues.

I. Qualitative Examples and Linked Fixes

Representative examples clarify the label boundaries. A CuPy issue titled "Wrong overflow with matmul of uint16 arrays" is a direct overflow_underflow case. A Triton issue involving remquo argument types is labeled dtype_casting because the failure centers on dtype signatures rather than numeric tolerance. A RAPIDS cuDF question about a Python type error is labeled not_numerical_failure, even though it contains type language, because the issue is an API question rather than a GPU correctness failure.

The artifact also contains linked-fix evidence for future root-cause work. In the pilot benchmark context, 105 issues contain at least one fix or root-cause signal, including 39 explicit pull-request URLs. A follow-on public diff fetch retrieved 488 changed-file rows from linked PRs. We additionally adjudicated a 50-row root-cause subset with five fix-oriented labels: compiler/backend/runtime behavior, dtype or type semantics, numerical algorithm or tolerance, performance/resource behavior, and non-bug feature/documentation/support cases. To expand coverage without overstating provenance, we also provide a 200-row evidence-coded root-cause extension. It contains the 50 human-adjudicated rows, 55 linked-fix evidence-coded rows, and 95 issue-evidence rows. Its largest groups are non-bug/support cases (62), numerical algorithm or tolerance cases (52), and dtype/type semantics cases (47). The paper treats this as root-cause supervision and provenance for future work, not as 200 human-adjudicated root-cause labels.

VII. Discussion

A. Why the Expanded Benchmark Matters

The expanded gold set changes the research conclusion. On the original 191-issue pilot, full-coverage classification remained near 54%, and the most useful setting was abstention. With 1,191 human-labeled examples, full-coverage triage becomes viable: the deterministic ensemble reaches 73.8% accuracy, the standalone LLM reaches 80.5% held-out accuracy, and chronological evaluation reaches 76.6% accuracy with the best deterministic model. This suggests that the earlier ceiling was primarily a

data and label-quality limitation, not evidence that issue reports are inherently unclassifiable.

B. Why Keep Both the LLM and Ensemble

For a paper, the standalone LLM is now both the easiest deployable artifact and the strongest full-coverage held-out model: it is one model that maps issue text to a label. The deterministic ensemble remains useful as a transparent baseline and confidence mechanism. Its abstention behavior is valuable because ambiguous issue reports are common, and a practical triage assistant should know when to defer. The strongest framing is therefore a two-level system: a standalone LLM provides full-coverage semantic triage, while LLM/ensemble agreement provides a high-confidence review queue.

C. Practical Use Cases

GPU-NFBenchcan support three workflows. First, maintainers can prioritize issues by separating likely numerical defects from performance or API reports. Second, researchers can sample issue clusters for root-cause analysis, such as dtype promotion failures or precision-tolerance mismatches. Third, future debugging agents can use the benchmark as a supervised evaluation set before attempting harder tasks such as patch localization or reproducer generation.

VIII. Threats to Validity

Annotation quality. The expanded benchmark relies on human labels and a deterministic normalization pass. Although invalid labels were removed and all repairs are logged, the full 1,191-row benchmark has not yet been double-adjudicated. The 119-row audit shows strong A/B agreement, and the follow-on v2 pass applied 419 adjudication updates, but the remaining non-adjudicated rows can still contain boundary-label errors.

Repository bias. The benchmark is drawn from public GitHub issues in popular GPU-related repositories. It may underrepresent private industrial bugs, closed-source compiler failures, or hardware-specific issues that are not reported publicly.

Cross-project generalization. We report leave-one-repository-out results, but project sizes are uneven. Apache TVM has only 36 issues, while JAX and PyTorch have nearly 300 each. Future versions should add more projects and report time-based splits to test whether models generalize to new issue styles.

Label granularity. Some reports naturally span multiple categories. For example, dtype conversion can lead to overflow, and compiler crashes can hide underlying numerical defects. This paper uses one primary label for evaluation, but multi-label secondary-cause modeling is an important extension.

Issue-text limitation. Many issue reports lack minimal reproducers, environment metadata, or confirmed fixes. The benchmark classifies reports; it does not prove the actual root cause unless linked fix evidence is available.

IX. Conclusion

This paper presents GPU-NFBench, a 1,191-row human-labeled benchmark for GPU numerical failure reports. The expanded benchmark shows that reliable full-coverage triage is possible when weak keyword labels are replaced with human gold labels: a standalone FLAN-T5-base model reaches 80.5% held-out accuracy and 0.778 macro F1, a deterministic voting ensemble reaches 73.8% accuracy and 0.712 macro F1, chronological deterministic evaluation reaches 76.6% accuracy and 0.738 macro F1, and LLM/deterministic agreement reaches 89.3% accuracy at 68.3% coverage. Leave-one-repository-out evaluation reaches 66.8% accuracy, showing that cross-project transfer remains difficult. The remaining path to a stronger archival submission is clear: adjudicate more of the expanded benchmark, add more projects, and extend report-level classification into root-cause and fix-evidence tasks.

Artifact Availability

The artifact contains the processed benchmark, expanded gold labels, taxonomy-normalization report, model-training scripts, fine-tuning files, evaluation tables, generated reports, qualitative examples, linked-fix evidence, a data card, completed blind audit files, v2 adjudication files, LLM fine-tuning files, the 50-row human-adjudicated root-cause subset, the 200-row evidence-coded root-cause extension, cross-repository weakness analysis, local LLM baseline outputs, and a 12-case error appendix. The data card documents intended use, non-use cases, label provenance, and known limitations. The release metadata includes a Zenodo-ready .zenodo.json and CITATION.cff; after GitHub authentication is restored, the artifact should be published as a tagged GitHub release and archived on Zenodo to obtain a DOI.

References

[1] P. Tillet, H. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, 2019.

[2] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in Proceedings of Workshop on Machine Learning Systems at NeurIPS, 2017.

[3] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems, 2019.

[4] J. Bradbury et al., "JAX: Composable transformations of Python+NumPy programs," 2018. [Online]. Available: https://github.com/jax-ml/jax

[5] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 2015.

[6] RAPIDS Development Team, "RAPIDS: Collection of GPU accelerated data science libraries," 2024. [Online]. Available: https://rapids.ai/

[7] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in 13th USENIX Symposium on Operating Systems Design and Implementation, 2018.

[8] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," Journal of Machine Learning Research, vol. 21, no. 140, pp. 1–67, 2020.

[9] H. W. Chung et al., “Scaling instruction-finetuned language models,” J. Mach. Learn. Res., vol. 25, no. 70, pp. 1–53, 2024.